Localizing_objects

April 11, 2026

[1]:
```
!pip install -U opencv-python==4.2.0.34
```

```
Requirement already up-to-date: opencv-python==4.2.0.34 in
/Users/ben/anaconda3/lib/python3.6/site-packages (4.2.0.34)
Requirement already satisfied, skipping upgrade: numpy>=1.11.3 in
/Users/ben/anaconda3/lib/python3.6/site-packages (from opencv-python==4.2.0.34)
(1.18.5)
```

[1]:
```
def download_file(url: str, filename='demo.jpg'):
    import requests
    response = requests.get(url)
    with open(filename, 'wb') as f:
        f.write(response.content)
```

[2]:
```
download_file('https://raw.githubusercontent.com/open-mmlab/mmdetection/master/
↪demo/demo.jpg')
```

[3]:
```
ls -laht demo.jpg
```

```
 Volume in drive C is Windows
 Volume Serial Number is A663-7E31

 Directory of C:\ZDrive\Yathish\Potomac\AIT 620\LabWorks\PythonProject\chapter08


 Directory of C:\ZDrive\Yathish\Potomac\AIT 620\LabWorks\PythonProject\chapter08

04/11/2026  02:11 PM           259,865 demo.jpg
               1 File(s)        259,865 bytes
               0 Dir(s)  652,757,995,520 bytes free
```

[4]:
```
download_file('https://www.dropbox.com/s/qhdbu7w3bi0tmyt/yolo.weights', 'yolo.
↪weights')
```

[5]:
```
ls -laht yolo.weights
```

```
 Volume in drive C is Windows
 Volume Serial Number is A663-7E31
```

```
 Directory of C:\ZDrive\Yathish\Potomac\AIT 620\LabWorks\PythonProject\chapter08


 Directory of C:\ZDrive\Yathish\Potomac\AIT 620\LabWorks\PythonProject\chapter08

04/11/2026  02:12 PM            98,793 yolo.weights
               1 File(s)         98,793 bytes
               0 Dir(s)  652,756,267,008 bytes free
```

[6]:

```python
import os
import numpy as np
from tensorflow.keras.layers import Conv2D, Input, BatchNormalization,␣
↪LeakyReLU, ZeroPadding2D, UpSampling2D
from tensorflow.keras.layers import concatenate, add
from tensorflow.keras.models import Model
import struct
import cv2

#np.set_printoptions(threshold=np.nan)
os.environ["CUDA_DEVICE_ORDER"]="PCI_BUS_ID"
os.environ["CUDA_VISIBLE_DEVICES"]="0"

class WeightReader:
    def __init__(self, weight_file):
        with open(weight_file, 'rb') as w_f:
            major,    = struct.unpack('i', w_f.read(4))
            minor,    = struct.unpack('i', w_f.read(4))
            revision, = struct.unpack('i', w_f.read(4))

            if (major*10 + minor) >= 2 and major < 1000 and minor < 1000:
                w_f.read(8)
            else:
                w_f.read(4)

            transpose = (major > 1000) or (minor > 1000)

            binary = w_f.read()

        self.offset = 0
        self.all_weights = np.frombuffer(binary, dtype='float32')

    def read_bytes(self, size):
        self.offset = self.offset + size
        return self.all_weights[self.offset-size:self.offset]

    def load_weights(self, model):
        for i in range(106):
```

```
            try:
                conv_layer = model.get_layer('conv_' + str(i))
                print("loading weights of convolution #" + str(i))

                if i not in [81, 93, 105]:
                    norm_layer = model.get_layer('bnorm_' + str(i))

                    size = np.prod(norm_layer.get_weights()[0].shape)

                    beta  = self.read_bytes(size) # bias
                    gamma = self.read_bytes(size) # scale
                    mean  = self.read_bytes(size) # mean
                    var   = self.read_bytes(size) # variance

                    weights = norm_layer.set_weights([gamma, beta, mean, var])

                if len(conv_layer.get_weights()) > 1:
                    bias   = self.read_bytes(np.prod(conv_layer.
 ↪get_weights()[1].shape))
                    kernel = self.read_bytes(np.prod(conv_layer.
 ↪get_weights()[0].shape))

                    kernel = kernel.reshape(list(reversed(conv_layer.
 ↪get_weights()[0].shape)))
                    kernel = kernel.transpose([2,3,1,0])
                    conv_layer.set_weights([kernel, bias])
                else:
                    kernel = self.read_bytes(np.prod(conv_layer.
 ↪get_weights()[0].shape))
                    kernel = kernel.reshape(list(reversed(conv_layer.
 ↪get_weights()[0].shape)))
                    kernel = kernel.transpose([2,3,1,0])
                    conv_layer.set_weights([kernel])
            except ValueError:
                print("no convolution #" + str(i))

    def reset(self):
        self.offset = 0

class BoundBox:
    def __init__(self, xmin, ymin, xmax, ymax, objness = None, classes = None):
        self.xmin = xmin
        self.ymin = ymin
        self.xmax = xmax
        self.ymax = ymax

        self.objness = objness
```

```
        self.classes = classes

        self.label = -1
        self.score = -1

    def get_label(self):
        if self.label == -1:
            self.label = np.argmax(self.classes)

        return self.label

    def get_score(self):
        if self.score == -1:
            self.score = self.classes[self.get_label()]

        return self.score

def _conv_block(inp, convs, skip=True):
    x = inp
    count = 0

    for conv in convs:
        if count == (len(convs) - 2) and skip:
            skip_connection = x
        count += 1

        if conv['stride'] > 1: x = ZeroPadding2D(((1,0),(1,0)))(x) # peculiar␣
 ↪padding as darknet prefer left and top
        x = Conv2D(conv['filter'],
                   conv['kernel'],
                   strides=conv['stride'],
                   padding='valid' if conv['stride'] > 1 else 'same', #␣
 ↪peculiar padding as darknet prefer left and top
                   name='conv_' + str(conv['layer_idx']),
                   use_bias=False if conv['bnorm'] else True)(x)
        if conv['bnorm']: x = BatchNormalization(epsilon=0.001, name='bnorm_' +␣
 ↪str(conv['layer_idx']))(x)
        if conv['leaky']: x = LeakyReLU(alpha=0.1, name='leaky_' +␣
 ↪str(conv['layer_idx']))(x)

    return add([skip_connection, x]) if skip else x

def _interval_overlap(interval_a, interval_b):
    x1, x2 = interval_a
    x3, x4 = interval_b

    if x3 < x1:
```

```
        if x4 < x1:
            return 0
        else:
            return min(x2,x4) - x1
    else:
        if x2 < x3:
             return 0
        else:
            return min(x2,x4) - x3

def _sigmoid(x):
    return 1. / (1. + np.exp(-x))

def bbox_iou(box1, box2):
    intersect_w = _interval_overlap([box1.xmin, box1.xmax], [box2.xmin, box2.
 ↪xmax])
    intersect_h = _interval_overlap([box1.ymin, box1.ymax], [box2.ymin, box2.
 ↪ymax])

    intersect = intersect_w * intersect_h

    w1, h1 = box1.xmax-box1.xmin, box1.ymax-box1.ymin
    w2, h2 = box2.xmax-box2.xmin, box2.ymax-box2.ymin

    union = w1*h1 + w2*h2 - intersect

    return float(intersect) / union

def make_yolov3_model():
    input_image = Input(shape=(None, None, 3))

    # Layer  0 => 4
    x = _conv_block(input_image, [{'filter': 32, 'kernel': 3, 'stride': 1,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 0},
                                  {'filter': 64, 'kernel': 3, 'stride': 2,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 1},
                                  {'filter': 32, 'kernel': 1, 'stride': 1,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 2},
                                  {'filter': 64, 'kernel': 3, 'stride': 1,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 3}])

    # Layer  5 => 8
    x = _conv_block(x, [{'filter': 128, 'kernel': 3, 'stride': 2, 'bnorm':␣
 ↪True, 'leaky': True, 'layer_idx': 5},
                        {'filter':  64, 'kernel': 1, 'stride': 1, 'bnorm':␣
 ↪True, 'leaky': True, 'layer_idx': 6},
```

```
                        {'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 7}])

   # Layer  9 => 11
   x = _conv_block(x, [{'filter':  64, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 9},
                        {'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 10}])

   # Layer 12 => 15
   x = _conv_block(x, [{'filter': 256, 'kernel': 3, 'stride': 2, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 12},
                        {'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 13},
                        {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 14}])

   # Layer 16 => 36
   for i in range(7):
       x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 16+i*3},
                           {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 17+i*3}])

   skip_36 = x

   # Layer 37 => 40
   x = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 2, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 37},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 38},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 39}])

   # Layer 41 => 61
   for i in range(7):
       x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 41+i*3},
                           {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 42+i*3}])

   skip_61 = x

   # Layer 62 => 65
   x = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 2, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 62},
```

```
                        {'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 63},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 64}])

    # Layer 66 => 74
    for i in range(3):
        x = _conv_block(x, [{'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 66+i*3},
                            {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 67+i*3}])

    # Layer 75 => 79
    x = _conv_block(x, [{'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 75},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 76},
                        {'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 77},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 78},
                        {'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 79}], skip=False)

    # Layer 80 => 82
    yolo_82 = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 80},
                              {'filter':  255, 'kernel': 1, 'stride': 1,␣
↪'bnorm': False, 'leaky': False, 'layer_idx': 81}], skip=False)

    # Layer 83 => 86
    x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 84}], skip=False)
    x = UpSampling2D(2)(x)
    x = concatenate([x, skip_61])

    # Layer 87 => 91
    x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 87},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 88},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 89},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 90},
```

```
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 91}], skip=False)

    # Layer 92 => 94
    yolo_94 = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':
↪ True,  'leaky': True,  'layer_idx': 92},
                              {'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm':
↪ False, 'leaky': False, 'layer_idx': 93}], skip=False)

    # Layer 95 => 98
    x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True,   'layer_idx': 96}], skip=False)
    x = UpSampling2D(2)(x)
    x = concatenate([x, skip_36])

    # Layer 99 => 106
    yolo_106 = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 99},
                               {'filter': 256, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 100},
                               {'filter': 128, 'kernel': 1, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 101},
                               {'filter': 256, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 102},
                               {'filter': 128, 'kernel': 1, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 103},
                               {'filter': 256, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 104},
                               {'filter': 255, 'kernel': 1, 'stride': 1,␣
↪'bnorm': False, 'leaky': False, 'layer_idx': 105}], skip=False)

    model = Model(input_image, [yolo_82, yolo_94, yolo_106])
    return model

def preprocess_input(image, net_h, net_w):
    new_h, new_w, _ = image.shape

    # determine the new size of the image
    if (float(net_w)/new_w) < (float(net_h)/new_h):
        new_h = (new_h * net_w)/new_w
        new_w = net_w
    else:
        new_w = (new_w * net_h)/new_h
        new_h = net_h

    # resize the image to the new size
```

```
    resized = cv2.resize(image[:,:,::-1]/255., (int(new_w), int(new_h)))

    # embed the image into the standard letter box
    new_image = np.ones((net_h, net_w, 3)) * 0.5
    new_image[int((net_h-new_h)//2):int((net_h+new_h)//2), int((net_w-new_w)//
↪2):int((net_w+new_w)//2), :] = resized
    new_image = np.expand_dims(new_image, 0)

    return new_image

def decode_netout(netout, anchors, obj_thresh, nms_thresh, net_h, net_w):
    grid_h, grid_w = netout.shape[:2]
    nb_box = 3
    netout = netout.reshape((grid_h, grid_w, nb_box, -1))
    nb_class = netout.shape[-1] - 5

    boxes = []

    netout[..., :2]  = _sigmoid(netout[..., :2])
    netout[..., 4:]  = _sigmoid(netout[..., 4:])
    netout[..., 5:]  = netout[..., 4][..., np.newaxis] * netout[..., 5:]
    netout[..., 5:] *= netout[..., 5:] > obj_thresh

    for i in range(grid_h*grid_w):
        row = i / grid_w
        col = i % grid_w

        for b in range(nb_box):
            # 4th element is objectness score
            objectness = netout[int(row)][int(col)][b][4]
            #objectness = netout[..., :4]

            if(objectness.all() <= obj_thresh): continue

            # first 4 elements are x, y, w, and h
            x, y, w, h = netout[int(row)][int(col)][b][:4]

            x = (col + x) / grid_w # center position, unit: image width
            y = (row + y) / grid_h # center position, unit: image height
            w = anchors[2 * b + 0] * np.exp(w) / net_w # unit: image width
            h = anchors[2 * b + 1] * np.exp(h) / net_h # unit: image height

            # last elements are class probabilities
            classes = netout[int(row)][col][b][5:]

            box = BoundBox(x-w/2, y-h/2, x+w/2, y+h/2, objectness, classes)
            #box = BoundBox(x-w/2, y-h/2, x+w/2, y+h/2, None, classes)
```

```
            boxes.append(box)

    return boxes

def correct_yolo_boxes(boxes, image_h, image_w, net_h, net_w):
    if (float(net_w)/image_w) < (float(net_h)/image_h):
        new_w = net_w
        new_h = (image_h*net_w)/image_w
    else:
        new_h = net_w
        new_w = (image_w*net_h)/image_h

    for i in range(len(boxes)):
        x_offset, x_scale = (net_w - new_w)/2./net_w, float(new_w)/net_w
        y_offset, y_scale = (net_h - new_h)/2./net_h, float(new_h)/net_h

        boxes[i].xmin = int((boxes[i].xmin - x_offset) / x_scale * image_w)
        boxes[i].xmax = int((boxes[i].xmax - x_offset) / x_scale * image_w)
        boxes[i].ymin = int((boxes[i].ymin - y_offset) / y_scale * image_h)
        boxes[i].ymax = int((boxes[i].ymax - y_offset) / y_scale * image_h)

def do_nms(boxes, nms_thresh):
    if len(boxes) > 0:
        nb_class = len(boxes[0].classes)
    else:
        return

    for c in range(nb_class):
        sorted_indices = np.argsort([-box.classes[c] for box in boxes])

        for i in range(len(sorted_indices)):
            index_i = sorted_indices[i]

            if boxes[index_i].classes[c] == 0: continue

            for j in range(i+1, len(sorted_indices)):
                index_j = sorted_indices[j]

                if bbox_iou(boxes[index_i], boxes[index_j]) >= nms_thresh:
                    boxes[index_j].classes[c] = 0

def draw_boxes(image, boxes, labels, obj_thresh):
    for box in boxes:
        label_str = ''
        label = -1
```

```
        for i in range(len(labels)):
            if box.classes[i] > obj_thresh:
                label_str += labels[i]
                label = i
                print(labels[i] + ': ' + str(box.classes[i]*100) + '%')

        if label >= 0:
            cv2.rectangle(image, (box.xmin,box.ymin), (box.xmax,box.ymax),␣
↪(0,255,0), 3)
            cv2.putText(image,
                        label_str + ' ' + str(box.get_score()),
                        (box.xmin, box.ymin - 13),
                        cv2.FONT_HERSHEY_SIMPLEX,
                        1e-3 * image.shape[0],
                        (0,255,0), 2)

    return image

def load_model(weights_path: str):
    # make the yolov3 model to predict 80 classes on COCO
    yolov3 = make_yolov3_model()

    # load the weights trained on COCO into the model
    weight_reader = WeightReader(weights_path)
    weight_reader.load_weights(yolov3)
    return yolov3

def detect(yolov3, image_path):
    """
    Detect objects and apply the yolo3 model

    Parameters:
    -----------
    yolov3 - keras yolo model
    image_path - either a filepath or an image
    """
    # set some parameters
    net_h, net_w = 416, 416
    obj_thresh, nms_thresh = 0.5, 0.45
    anchors = [[116,90,  156,198,  373,326],  [30,61, 62,45,  59,119], [10,13, ␣
↪16,30,  33,23]]
    labels = ["person", "bicycle", "car", "motorbike", "aeroplane", "bus",␣
↪"train", "truck", \
              "boat", "traffic light", "fire hydrant", "stop sign", "parking␣
↪meter", "bench", \
              "bird", "cat", "dog", "horse", "sheep", "cow", "elephant",␣
↪"bear", "zebra", "giraffe", \
```

```
            "backpack", "umbrella", "handbag", "tie", "suitcase", "frisbee",␣
↪"skis", "snowboard", \
            "sports ball", "kite", "baseball bat", "baseball glove",␣
↪"skateboard", "surfboard", \
            "tennis racket", "bottle", "wine glass", "cup", "fork", "knife",␣
↪"spoon", "bowl", "banana", \
            "apple", "sandwich", "orange", "broccoli", "carrot", "hot dog",␣
↪"pizza", "donut", "cake", \
            "chair", "sofa", "pottedplant", "bed", "diningtable", "toilet",␣
↪"tvmonitor", "laptop", "mouse", \
            "remote", "keyboard", "cell phone", "microwave", "oven",␣
↪"toaster", "sink", "refrigerator", \
            "book", "clock", "vase", "scissors", "teddy bear", "hair drier",␣
↪"toothbrush"]

    # preprocess the image
    if isinstance(image_path, str):
        image = cv2.imread(image_path)
    else:
        image = image_path
    image_h, image_w, _ = image.shape
    new_image = preprocess_input(image, net_h, net_w)

    # run the prediction
    yolos = yolov3.predict(new_image)
    boxes = []

    for i in range(len(yolos)):
        # decode the output of the network
        boxes += decode_netout(
            yolos[i][0],
            anchors[i],
            obj_thresh,
            nms_thresh,
            net_h,
            net_w
        )

    # correct the sizes of the bounding boxes
    correct_yolo_boxes(boxes, image_h, image_w, net_h, net_w)

    # suppress non-maximal boxes
    do_nms(boxes, nms_thresh)

    # draw bounding boxes on the image using labels
    draw_boxes(image, boxes, labels, obj_thresh)
```

```
    plt.figure(figsize = (15,10))
    return image
    #plt.imshow(image)
    # write the image with bounding boxes to file
    # cv2.imwrite(image_path[:-4] + '_detected' + image_path[-4:], (image).
↪astype('uint8'))
```

[7]:
```
download_file('https://pjreddie.com/media/files/yolov3.weights', 'yolov3.
↪weights')
```

[8]:
```
yolov3 = load_model('yolov3.weights')
```

```
C:\Users\yathi\anaconda3\Lib\site-
packages\keras\src\layers\activations\leaky_relu.py:41: UserWarning: Argument
`alpha` is deprecated. Use `negative_slope` instead.
  warnings.warn(

loading weights of convolution #0
loading weights of convolution #1
loading weights of convolution #2
loading weights of convolution #3
no convolution #4
loading weights of convolution #5
loading weights of convolution #6
loading weights of convolution #7
no convolution #8
loading weights of convolution #9
loading weights of convolution #10
no convolution #11
loading weights of convolution #12
loading weights of convolution #13
loading weights of convolution #14
no convolution #15
loading weights of convolution #16
loading weights of convolution #17
no convolution #18
loading weights of convolution #19
loading weights of convolution #20
no convolution #21
loading weights of convolution #22
loading weights of convolution #23
no convolution #24
loading weights of convolution #25
loading weights of convolution #26
no convolution #27
loading weights of convolution #28
loading weights of convolution #29
no convolution #30
```

```
loading weights of convolution #31
loading weights of convolution #32
no convolution #33
loading weights of convolution #34
loading weights of convolution #35
no convolution #36
loading weights of convolution #37
loading weights of convolution #38
loading weights of convolution #39
no convolution #40
loading weights of convolution #41
loading weights of convolution #42
no convolution #43
loading weights of convolution #44
loading weights of convolution #45
no convolution #46
loading weights of convolution #47
loading weights of convolution #48
no convolution #49
loading weights of convolution #50
loading weights of convolution #51
no convolution #52
loading weights of convolution #53
loading weights of convolution #54
no convolution #55
loading weights of convolution #56
loading weights of convolution #57
no convolution #58
loading weights of convolution #59
loading weights of convolution #60
no convolution #61
loading weights of convolution #62
loading weights of convolution #63
loading weights of convolution #64
no convolution #65
loading weights of convolution #66
loading weights of convolution #67
no convolution #68
loading weights of convolution #69
loading weights of convolution #70
no convolution #71
loading weights of convolution #72
loading weights of convolution #73
no convolution #74
loading weights of convolution #75
loading weights of convolution #76
loading weights of convolution #77
loading weights of convolution #78
```

```
loading weights of convolution #79
loading weights of convolution #80
loading weights of convolution #81
no convolution #82
no convolution #83
loading weights of convolution #84
no convolution #85
no convolution #86
loading weights of convolution #87
loading weights of convolution #88
loading weights of convolution #89
loading weights of convolution #90
loading weights of convolution #91
loading weights of convolution #92
loading weights of convolution #93
no convolution #94
no convolution #95
loading weights of convolution #96
no convolution #97
no convolution #98
loading weights of convolution #99
loading weights of convolution #100
loading weights of convolution #101
loading weights of convolution #102
loading weights of convolution #103
loading weights of convolution #104
loading weights of convolution #105
```

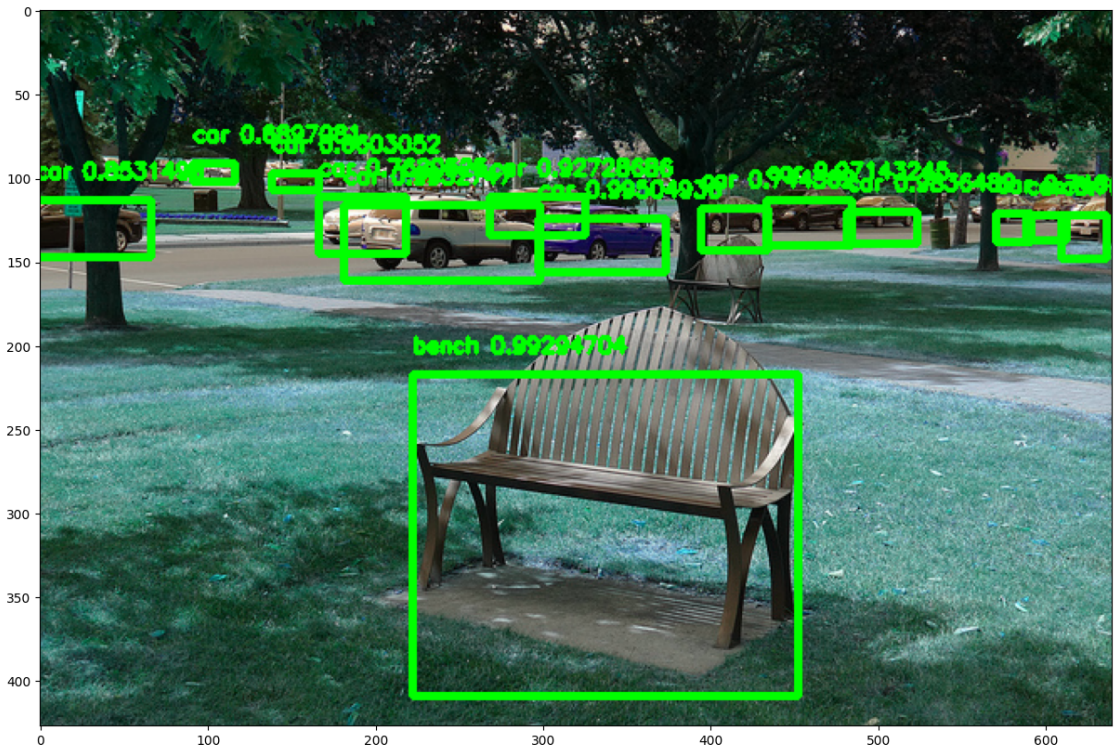
[9]:
```
from matplotlib import pyplot as plt

plt.imshow(detect(yolov3, 'demo.jpg'))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.
Please use WSL2 or the TensorFlow-DirectML plugin.
1/1             3s 3s/step
bench: 99.2947%
car: 99.32747%
car: 68.97081%
car: 66.03052%
car: 76.205246%
car: 92.72868%
car: 74.4562%
car: 97.14324%
car: 98.36489%
car: 74.619865%
car: 76.97725%
car: 90.979095%
```

```
car: 85.31498%
car: 99.50494%
```

[9]: <matplotlib.image.AxesImage at 0x25206ab16a0>



[14]:
```python
import cv2
from skimage import color

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Convert grayscale back to BGR
    img = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

    # Run YOLO detection (expects BGR)
    detected = detect(yolov3, img)
```

```
    # detect() returns RGB → convert back to BGR for OpenCV
    detected_bgr = cv2.cvtColor(detected, cv2.COLOR_RGB2BGR)

    cv2.imshow('YOLO Webcam', detected_bgr)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

```
1/1              0s 349ms/step
person: 99.972015%
1/1              0s 294ms/step
person: 99.98546%
1/1              0s 343ms/step
person: 99.98232%
1/1              0s 313ms/step
person: 99.987915%
1/1              0s 324ms/step
person: 98.6421%
person: 78.65937%
1/1              0s 310ms/step
person: 99.98113%
1/1              0s 328ms/step
person: 99.98074%
1/1              0s 322ms/step
person: 99.97526%
1/1              0s 332ms/step
person: 99.973755%
```

```
<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>

<Figure size 1500x1000 with 0 Axes>
```

[ ]:

```

```